Kuku Localise: Series-Aware Microdrama Localisation

Action Plan Refactoring & Architecture Report

1. Honesty of Measurement

Evaluates model_display_name (the raw model output) to honestly measure cue consistency before code-enforced canonical normalisation. Dynamic KPI badges in the studio continuity scorecard are directly bound to missing_beat_count and timestamps_preserved data.

2. Pipeline Architecture & Principle Enforcement

Axiom: Give the LLM only judgment work; keep measurable constraints in plain code.

- Step 1: Enhanced parser extracts clean cue names, strips nested parentheticals and audio modifiers (O.S., V.O.), and calculates beat duration in seconds.
- Step 2: Expanded Story Bible extracts gender (crucial for Hindi verb agreement), formal/informal address registers (aap/tum/tu), and surfaces low-confidence alias merges for human review.
- Step 3: Context-rich translation prompt injects recent dialogue history, scene context, and time budgets.
- Step 4: Strict code validation evaluates characters-per-second (CPS <= 18.0), Unicode address register compliance, and vertical screen subtitle limits.
- Step 5: Automated repair loop retries failing beats with explicit error directives (capped at 2 retries), tagging persistent violations as Honest Failures for human intervention.
- Step 6: Secondary cultural adaptation pass identifies Western/US idioms and attaches localized equivalents.

3. Real Evidence Metrics (Studio ROI)

Replaces purely structural checks with the Reviewer Edit Rate: (characters changed / characters delivered). Computes Levenshtein edit distance per beat and reports Reviewer Effort Saved (%) comparing series contextual translations against isolated baselines.

4. Model Swappability

Decoupled from Gemini via BaseLLMClient abstract interface. Powered by get_client(provider, model) factory supporting GeminiClient, MockLLMClient, and configurable fallbacks (gemini-2.5-flash).

5. Scale and Cost

Provides --no-baseline flag to cut production API calls and costs by 50%. Aggregates API calls, prompt/candidate tokens, and estimated USD costs in usage.json. The Story Bible is generated once per show, and responses are deterministically cached.